

COURSE CODE/TITLE:	CPE 212 - Automating Server Management	SECTION:	CPE31S2
DATE PERFORMED:	09/10/2026	DATE SUBMITTED:	09/10/2026
NAME:	Interno, Jerald San M.	INSTRUCTOR:	Engr. Jimlord Quejado

ACTIVITY NO. ____	[ACTIVITY NAME]
--------------------------	------------------------

1. PROCEDURE OUTPUT

What procedures did you perform?

Task 1: Create a file and copy it to remote servers

1. Using the previous directory was created in your previous laboratory, create a directory, and name it "files." Create a file inside that directory and name it "default_site.html." Edit the file and put basic HTML syntax. Any content will do, as long as it will display text later. Save the file and exit.

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>My Web Server</title>
</head>
<body>
<h1>Welcome to My Web Server</h1>
<p>This webpage was deployed using Ansible.</p>
<p>This file was copied from the local repository to the remote server.</p>
</body>
</html>
```

2. Edit the site.yml file and just below the web_servers play, create a new file to copy the default html file for site:

```
- name: copy default html file for site
tags: apache, apache2, httpd
copy:
src: files/default_site.html
dest: /var/www/html/index.html
owner: root
group: root
mode: '0644'
```

3. Run the playbook site.yml. Describe the changes.
None

4. Go to the remote servers (web_servers) listed in your inventory. Use cat command to check if the index.html is the same as the local repository file (default_site.html). Do both for Ubuntu and CentOS servers. On the CentOS server, go to the browser and type its IP address. Describe the output.

5. Sync your local repository with GitHub and describe the changes.

```
jerald@workstation:~/CPE212_Jerald_Interno$ mkdir mkdir -p files
jerald@workstation:~/CPE212_Jerald_Interno$ nano files/default_site.html
```

```
jeraldinterno@localhost:~
GNU nano 7.2 files/default_site.html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Web Server</title>
</head>
<body>
  <h1>Welcome to My Web Server</h1>
  <p>This webpage was deployed using Ansible.</p>
  <p>This file was copied from the local repository to the remote server.</p>
</body>
</html>
```

```
jerald@workstation:~/CPE212_Jerald_Interno$ nano site.yml
```

```
jeraldinterno@localhost:~
GNU nano 7.2 site.yml
- hosts: web_servers
  become: true
  tasks:
    # ... (keep existing install tasks) ...

    - name: copy default html file for site
      tags: [apache, apache2, httpd]
      copy:
        src: files/default_site.html
        dest: /var/www/html/index.html
        owner: root
        group: root
        mode: '0644'
```

```
jerald@workstation:~/CPE212_Jerald_Interno$ ansible-playbook -i inventory.yaml site.yml -k -K
SSH password:
BECOME password[defaults to SSH password]:

PLAY [web_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.104]
ok: [192.168.56.101]
ok: [192.168.56.103]

TASK [copy default html file for site] *****
changed: [192.168.56.104]
changed: [192.168.56.103]
changed: [192.168.56.101]

PLAY RECAP *****
192.168.56.101      : ok=2    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
192.168.56.103      : ok=2    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
192.168.56.104      : ok=2    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

```
jerald@workstation:~/CPE212_Jerald_Interno$ ssh interno@192.168.56.103 "cat /var/www/html/index.html"
interno@192.168.56.103's password:
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Web Server</title>
</head>
<body>
  <h1>Welcome to My Web Server</h1>
  <p>This webpage was deployed using Ansible.</p>
  <p>This file was copied from the local repository to the remote server.</p>
</body>
</html>
```

```
jerald@workstation:~/CPE212_Jerald_Interno$ ssh jeraldinterno@192.168.56.101 "cat /var/www/html/index.html"

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Web Server</title>
</head>
<body>
  <h1>Welcome to My Web Server</h1>
  <p>This webpage was deployed using Ansible.</p>
  <p>This file was copied from the local repository to the remote server.</p>
</body>
</html>
```

192.168.56.101

← → ↻ Not Secure http://192.168.56.101 ☆ 🔒 ⌵

Welcome to My Web Server

This webpage was deployed using Ansible.

This file was copied from the local repository to the remote server.

Task 2: Download a file and extract it to a remote server

1. Edit the site.yml. Just before the web_servers play, create a new play:

- hosts: workstations

become: true

tasks:

- name: install unzip

package:

name: unzip

- name: install terraform

unarchive:

src: https://releases.hashicorp.com/terraform/0.12.28/terraform_0.12.28_linux_amd64.zip

dest: /usr/local/bin

remote_src: yes

mode: 0755

owner: root

group: root

note: In Ansible, a play is a section of a playbook that defines which remote machines Ansible should manage and what tasks it should perform on them.

2. Edit the inventory file and add a workstation group. Add any Ubuntu remote server. Make sure to remember the IP address.

3. Run the playbook. Describe the output.

4. On the Ubuntu remote workstation, type terraform to verify installation of terraform. Describe the output.

- **Ansible successfully installed unzip as a prerequisite before using the unarchive module with remote_src: yes to download and extract Terraform directly to /usr/local/bin on the target workstation. Applying file permissions (0755) ensured the binary was immediately executable system-wide, which was verified on the remote node when terraform --version returned Terraform v0.12.28.**

```
jerald@workstation:~/CPE212_Jerald_Interno$ nano inventory.yml
```

```
internos2@server2: ~
GNU nano 7.2 inventory.yaml
[web_servers]
192.168.56.101 ansible_user=jeraldinterno
192.168.56.103 ansible_user=interno
192.168.56.104 ansible_user=internos2

[db_servers]
192.168.56.101 ansible_user=jeraldinterno

[file_servers]
192.168.56.103 ansible_user=interno

[workstations]
192.168.56.104 ansible_user=internos2
```

jerald@workstation:~/CPE212_Jerald_Interno\$ nano site.yml

```
jeraldinterno@localhost:~
GNU nano 7.2 site.yml
- hosts: workstations
  become: true
  tasks:
    - name: install unzip
      package:
        name: unzip

    - name: install terraform
      unarchive:
        src: https://releases.hashicorp.com/terraform/0.12.28/terraform-0.12.28-linux-amd64.zip
        dest: /usr/local/bin
        remote_src: yes
        mode: '0755'
        owner: root
        group: root
```

```
jerald@workstation:~/CPE212_Jerald_Interno$ ansible-playbook -i inventory.yaml site.yaml -k -K
SSH password:
BECOME password[defaults to SSH password]:

PLAY [workstations] *****

TASK [Gathering Facts] *****
ok: [192.168.56.104]

TASK [install unzip] *****
ok: [192.168.56.104]

TASK [install terraform] *****
changed: [192.168.56.104]

PLAY RECAP *****
192.168.56.104 : ok=3    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

```
jerald@workstation:~/CPE212_Jerald_Interno$ ssh internos2@192.168.56.104
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 7.0.0-30-generic x86_64)
```

```
* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/pro
```

Expanded Security Maintenance for Applications is not enabled.

47 updates can be applied immediately.

To see these additional updates run: `apt list --upgradable`

Enable ESM Apps to receive additional future security updates.

See <https://ubuntu.com/esm> or run: `sudo pro status`

*** System restart required ***

Last login: Thu Sep 10 11:04:38 2026 from 192.168.56.1

```
internos2@server2:~$ terraform --version
```

Terraform v0.12.28

Your version of Terraform is out of date! The latest version

is 1.16.2. You can update by downloading from <https://www.terraform.io/downloads.html>

Task 3: Create roles

1. Edit the site.yaml. Configure roles as follows: (make sure to create a copy of the old site.yaml file because you will be copying the specific plays for all groups)
Save the file and exit.
2. Under the same directory, create a new directory and name it roles. Enter the roles directory and create new directories: base, web_servers, file_servers, db_servers and workstations. For each directory, create a directory and name it tasks.
3. Go to tasks for all directory and create a file. Name it main.yaml. In each of the tasks for all directories, copy and paste the code from the old site.yaml file. Show all contents of main.yaml files for all tasks.
4. Run the site.yaml playbook and describe the output.

```
jerald@workstation:~/CPE212_Jerald_Interno$ cp site.yaml site.yaml.bak
```

```
jerald@workstation:~/CPE212_Jerald_Interno$ mkdir -p roles/{base,web_servers,file_servers,db_servers,workstations}/tasks
```

```
jerald@workstation:~/CPE212_Jerald_Interno$ nano roles/base/tasks/main.yaml
```

```
internos2@server2: ~  
GNU nano 7.2 roles/base/tasks/main.yml *  
---  
- name: install updates (CentOS)  
  dnf:  
    update_only: yes  
    update_cache: yes  
  when: ansible_distribution == "CentOS"  
  
- name: install updates (Ubuntu)  
  apt:  
    update_cache: yes  
  when: ansible_distribution == "Ubuntu"
```

jerald@workstation:~/CPE212_Jerald_Interno\$ nano roles/web_servers/tasks/main.yml

```
internos2@server2: ~  
GNU nano 7.2 roles/web_servers/tasks/main.yml *  
  
- name: start and enable httpd service on CentOS  
  service:  
    name: httpd  
    state: started  
    enabled: yes  
  when: ansible_distribution == "CentOS"  
  
- name: copy default html file for site  
  copy:  
    src: files/default_site.html  
    dest: /var/www/html/index.html  
    owner: root  
    group: root  
    mode: '0644'
```

jerald@workstation:~/CPE212_Jerald_Interno\$ nano roles/db_servers/tasks/main.yml

```
internos2@server2: ~
GNU nano 7.2 roles/db_servers/tasks/main.yml *
---
- name: install mariadb server (Ubuntu)
  apt:
    name: mariadb-server
    state: latest
  when: ansible_distribution == "Ubuntu"

- name: install mariadb server (CentOS)
  dnf:
    name: mariadb-server
    state: latest
  when: ansible_distribution == "CentOS"

jerald@workstation:~/CPE212_Jerald_Interno$ nano roles/file_servers/tasks/main.yml
Show Apps

internos2@server2: ~
GNU nano 7.2 roles/file_servers/tasks/main.yml *
---
- name: install samba package
  package:
    name: samba
    state: latest

jerald@workstation:~/CPE212_Jerald_Interno$ nano roles/workstations/tasks/main.yml
jerald@workstation:~/CPE212_Jerald_Interno$ nano site.yml

internos2@server2: ~
GNU nano 7.2 roles/workstations/tasks/main.yml *
---
- name: install unzip
  package:
    name: unzip

- name: install terraform
  unarchive:
    src: https://releases.hashicorp.com/terraform/0.12.28/terraform_0.12.28_linux_amd64.zip
    dest: /usr/local/bin
    remote_src: yes
    mode: '0755'
    owner: root
    group: root

jerald@workstation:~/CPE212_Jerald_Interno$ nano site.yml
```



internos2@server2: ~

GNU nano 7.2

site.yml *

```
--
- hosts: all
  become: true
  roles:
    - base

- hosts: web_servers
  become: true
  roles:
    - web_servers

- hosts: db_servers
  become: true
  roles:
    - db_servers

- hosts: file_servers
  become: true
  roles:
    - file_servers

- hosts: workstations
  become: true
  roles:
    - workstations
```



```
jerald@workstation:~/CPE212_Jerald_Interno$ ansible-playbook -i inventory.yaml site.yml -k -K
SSH password:
BECOME password[defaults to SSH password]:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [192.168.56.103]
ok: [192.168.56.104]
ok: [192.168.56.101]

TASK [base : install updates (CentOS)] *****
skipping: [192.168.56.103]
skipping: [192.168.56.104]
ok: [192.168.56.101]

TASK [base : install updates (Ubuntu)] *****
skipping: [192.168.56.101]
changed: [192.168.56.104]
changed: [192.168.56.103]

PLAY [web_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.104]
ok: [192.168.56.103]
ok: [192.168.56.101]

TASK [web_servers : install apache and php for Ubuntu servers] *****
skipping: [192.168.56.101]
ok: [192.168.56.104]
ok: [192.168.56.103]

TASK [web_servers : install apache and php for CentOS servers] *****
skipping: [192.168.56.103]
skipping: [192.168.56.104]
ok: [192.168.56.101]

TASK [web_servers : start and enable httpd service on CentOS] *****
skipping: [192.168.56.103]
skipping: [192.168.56.104]
ok: [192.168.56.101]

TASK [web_servers : copy default html file for site] *****
ok: [192.168.56.103]
ok: [192.168.56.104]
ok: [192.168.56.101]

PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.101]

TASK [db_servers : install mariadb server (Ubuntu)] *****
skipping: [192.168.56.101]

TASK [db_servers : install mariadb server (CentOS)] *****
ok: [192.168.56.101]

PLAY [file_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.103]
```

```
TASK [file_servers : install samba package] *****
ok: [192.168.56.103]

PLAY [workstations] *****

TASK [Gathering Facts] *****
ok: [192.168.56.104]

TASK [workstations : install unzip] *****
ok: [192.168.56.104]

TASK [workstations : install terraform] *****
ok: [192.168.56.104]

PLAY RECAP *****
192.168.56.101      : ok=8    changed=0    unreachable=0    failed=0    skipped=3    rescued=0    ignored=0
192.168.56.103      : ok=7    changed=1    unreachable=0    failed=0    skipped=3    rescued=0    ignored=0
192.168.56.104      : ok=8    changed=1    unreachable=0    failed=0    skipped=3    rescued=0    ignored=0
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Answer the following:

1. What is the importance of creating roles?

- In creating roles in Ansible organizes playbooks into modular and reusable directory structures. Instead of managing monolithic YAML files, the roles allows the administrators to decouple tasks, variables, handlers, and templates into discrete components based in function like web_servers and db_servers. In this modularity improves code readability, streamlines maintenance, prevents redundancy, and this allows roles to be easily shared and reused across the different projects and environments.

2. What is the importance of managing files?

- The importance of managing files through ansible ensures that consistent configuration state, file integrity, and security across all nodes in an infrastructure. Through utilizing the modules like copy and unarchive, the administrators can automates the distribution of custom content which are the web pages, application binaries, and configuration files while enforcing exact file permissions (mode), ownership (owner/group), and path destinations. Through this it eliminates configuration drift and removes the errors associated with manual file placement across servers.

Conclusions/Learnings:

- In this lab activity, key infrastructure automation techniques were implemented, including remote file distribution, automated binary deployment, and role-based architecture modularization. Utilizing the copy module made it simple to deploy web pages consistently, while the unarchive module streamlined software installations by fetching and extracting remote binaries directly on target hosts. By moving from the flat playbooks to structured Ansible Roles demonstrated the industry-standard configuration management practice. The grouping server tasks distinct roles like base, web_servers, db_servers, file_servers, and workstations provides a clean, organize, and simplified playbook orchestration. Overall, the lab activity teaches and highlighted how Ansible delivers scalable, maintainable, and predictable infrastructure management.



3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.